C socket API reference

Program Skeleton

```
#if defined _WIN32
#include <winsock2.h>
#include <ws2tcpip.h> /* link: ws2_32 */
#else
#define closesocket close
#include <sys/socket.h>
#include <arpa/inet.h>
#include <unistd.h>
#include <netdb.h>
#endif

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdint.h>
...
```

```
int main(void) {
...
      #if defined _WIN32 // initialize Winsock
      WSADATA wsa_data;
      int result =
         WSAStartup(MAKEWORD(2,2), &wsa_data);
      if (result != 0) {
         printf("Error at WSAStartup()\n");
        return EXIT_FAILURE;
      }
      #endif
      int my_socket;
...
      closesocket(my_socket);
      #if defined _WIN32
        WSACleanup();
      #endif
      return EXIT_SUCCESS;
} // main end
```

Data Structures

```
struct sockaddr {
        unsigned short  sa_family;
        char    sa_data[14];
};

struct sockaddr_in {
        unsigned short sin_family;
        unsigned short sin_port;
        struct  in_addr sin_addr;
        char    sin_zero[8];
};

struct in_addr { uint32_t s_addr; };
```

```
   struct addrinfo {
           int ai_flags;
           int ai_family;
           int ai_socktype;
           int ai_protocol;
           socklen_t ai_addrlen;
           struct sockaddr *ai_addr;
           char *ai_canonname;
           struct addrinfo *ai_next;
   };
    /* member order varies by OS */
```

Socket Setup

```
int socket(int pf, int type, int protocol);
int bind(int socket, const struct sockaddr* localaddress, int addr_len);
```

Socket Connection

```
int connect(int socket, const struct sockaddr* addr, int addrlen);
int listen(int socket, int backlog);
int accept(int socket, struct sockaddr* addr, int* addrlen);
```

Socket Communication

```
int send(int socket, const char* buf, int len, int flags);
int sendto(int s, const char* buf, int len, int flags, const struct sockaddr* to,
          unsigned int to_len);
int recv(int socket, char* buf, int len, int flags);
int recvfrom(int s, char* buf, int len, int flags, struct sockaddr* from,
             unsigned int* from_len);
int closesocket (int socket); /* Windows */
int close (int socket);       /* Unix */
/* Simplified signatures. POSIX: lengths are socklen_t, an unsigned type (accept/recvfrom: socklen_t*), buffers are
void*, sizes size_t, send/recv/sendto/recvfrom return ssize_t; on Windows lengths are int and a socket is a SOCKET */
```

Name/Address Resolution

```
int getaddrinfo(const char *node, const char *service,
                const struct addrinfo *hints, struct addrinfo **res);
void freeaddrinfo(struct addrinfo *res);
const char *gai_strerror(int errcode); // error message for getaddrinfo()/getnameinfo()
int getnameinfo(const struct sockaddr *sa, socklen_t salen, char *host, socklen_t hostlen,
                char *serv, socklen_t servlen, int flags); // flags: NI_NUMERICHOST, NI_NAMEREQD
```

Conversion functions

```
uint32_t inet_addr(const char* add); //dots-and-numbers string to 32-bit address (network byte order)
char* inet_ntoa(struct in_addr in); //struct in_addr to a dots-and-numbers string
uint16_t htons(uint16_t hs); //host to network short
uint32_t htonl(uint32_t hl); //host to network long
uint16_t ntohs(uint16_t ns); //network to host short
uint32_t ntohl(uint32_t nl); //network to host long
int atoi(const char* str); //string to integer
snprintf(str, size, "%d", value); // integer to string (portable; itoa() is non-standard)
```